

```
In [38]: from IPython.display import Image
         Image('k1-1.png')
```

Out[38]:

문제 6

(kaggle 형 문제) prob4 를 바탕으로 아래 데이터셋을 만든다.

. prob6_train: DateHour 변수 기준으로 2021년 8월 14일 전(8월 14일 미포함) 데이터 (행의 수: 5256개)

. prob6_test: DateHour 변수 기준으로 2021년 8월 14일 이후(8월 14일 포함) 데이터 (행의 수: 744 개)

일 때, prob6_train으로 target을 예측하는 모델을 만들어,

prob6_test의 target에 대한 MAE를 최소화하는 모델을 만든다.

prob6_test의 예측 결과를 아래와 같은 형식으로 출력한다. 파일명은 answer6.csv 이다.

DateHour	TotalHour
2021-08-14 00:00:00	102.607580
2021-08-14 01:00:00	94.078890
.....	

※ 실제 상황에 가까운 연습을 위해 prob6_test의 target 값은 모른다는 상황을 가정하고 문제를 푼다.

```
In [1]: # 실행 환경 확인

import pandas as pd
import numpy as np
import sklearn
import scipy
import statsmodels
#import mlxtend
import sys
#import xgboost as xgb

import warnings
```

```
warnings.filterwarnings(action = 'ignore')
```

```
print(sys.version)
```

```
for i in [pd, np, sklearn, scipy, statsmodels]:  
    print(i.__name__, i.__version__)
```

```
3.7.4 (tags/v3.7.4:e09359112e, Jul 8 2019, 20:34:20) [MSC v.1916 64 bit (AMD64)]
```

```
pandas 0.25.1
```

```
numpy 1.18.5
```

```
sklearn 0.21.3
```

```
scipy 1.5.2
```

```
statsmodels 0.11.1
```

```
In [2]: df_elec = pd.read_csv('elec.csv', parse_dates=['Date', 'DateHour'])  
df_info = pd.read_csv('info.csv')  
df_info['Date'] = pd.to_datetime(df_info['Date'])
```

```
In [3]: # index에 위치한 DateHour를 컬럼에 위치시킵니다
```

```
df_elec1 = df_elec.pivot(  
    index='DateHour',  
    columns='Minute',  
    values='Value').reset_index()
```

```
df_elec1
```

Out[3]:

	Minute	DateHour	15분	30분	45분	60분
--	--------	----------	-----	-----	-----	-----

0	2021-01-01 00:00:00	62	61	61	61
1	2021-01-01 01:00:00	96	93	116	113
2	2021-01-01 02:00:00	106	96	106	107
3	2021-01-01 03:00:00	92	110	110	109
4	2021-01-01 04:00:00	108	105	106	108
...	...	...	...	...	...
6163	2021-09-14 19:00:00	152	151	171	139
6164	2021-09-14 20:00:00	124	130	128	130
6165	2021-09-14 21:00:00	134	130	125	124
6166	2021-09-14 22:00:00	100	109	120	114
6167	2021-09-14 23:00:00	117	119	112	91

6168 rows × 5 columns

```
In [4]: '''
코드는 `df_elec1` 데이터프레임에 새로운 파생 변수를 추가하여 전기 사용량 데이터에 대한 다양한 시간 및 날짜 관련 정보를 계산하고 저장

### 동작 설명

1. 공휴일 목록 생성:
- `holi = pd.to_datetime(["2021-01-01", "2021-02-11", "2021-02-12", "2021-03-01", "2021-05-05", "2021-05-19", "2021-08-16"]).date`
- 지정된 날짜 문자열을 `datetime` 형식으로 변환한 후, `.date`를 사용하여 `datetime.date` 객체로 변환합니다.
- `holi`는 2021년의 특정 공휴일 날짜들을 포함하는 리스트입니다.

2. 파생 변수 생성:
- `df_elec1.assign(...)` 메서드를 사용하여 `df_elec1` 데이터프레임에 새로운 열을 추가합니다.

3. 파생 변수 설명:
- `DayName`:
- `df_elec1['DateHour'].dt.weekday`
- `DateHour` 열의 날짜에서 요일 번호를 추출합니다. (월요일 = 0, 일요일 = 6)
- `Hour`:
'''
```

```

- `df_elec1['DateHour'].dt.hour`
- `DateHour` 열에서 시간을 추출합니다.
- `AM` :
- `(df_elec1['DateHour'].dt.hour >= 12).astype('int')`
- 시간이 12시 이상이면 `1`, 아니면 `0`으로 변환합니다.
- `Weekend_yn` :
- `df_elec1['DateHour'].dt.weekday.isin([5, 6]).astype('int')`
- 요일이 토요일(5) 또는 일요일(6)이면 `1`, 아니면 `0`으로 변환합니다.
- `Holiday_yn` :
- `df_elec1['DateHour'].dt.date.isin(holi).astype('int')`
- `DateHour`의 날짜가 `holi` 리스트에 포함되어 있으면 `1`, 아니면 `0`으로 변환합니다.
- `Avg` :
- `df_elec1[['15분', '30분', '45분', '60분']].mean(axis=1)`
- 각 행의 `15분`, `30분`, `45분`, `60분` 열 값들의 평균을 계산합니다.
- `TotalHour` :
- `df_elec1[['15분', '30분', '45분', '60분']].sum(axis=1)`
- 각 행의 `15분`, `30분`, `45분`, `60분` 열 값들의 합을 계산합니다.

```

요약

코드는 `df\_elec1` 데이터프레임에 다음과 같은 새로운 열을 추가하여 시간 및 날짜 관련 정보를 포함합니다:

```

- `DayName` : 요일 번호
- `Hour` : 시간
- `AM` : 오전(0) 또는 오후(1) 표시
- `Weekend_yn` : 주말 여부
- `Holiday_yn` : 공휴일 여부
- `Avg` : 15분 간격 전기 사용량의 평균
- `TotalHour` : 15분 간격 전기 사용량의 합

```

이러한 파생 변수를 통해 데이터 분석 및 모델링 시 시간 및 날짜 관련 특성을 반영할 수 있습니다.

...

```
holi = pd.to_datetime(["2021-01-01", "2021-02-11", "2021-02-12", "2021-03-01", "2021-05-05", "2021-05-19", "2021-08-16"]).date
```

pd.Series.dt accessor를 통해 파생 변수들을 만듭니다.

```

df_elec1 = df_elec1.assign(
    DayName = df_elec1['DateHour'].dt.weekday,
    Hour = df_elec1['DateHour'].dt.hour,
    AM = (df_elec1['DateHour'].dt.hour >= 12).astype('int'),
    Weekend_yn = df_elec1['DateHour'].dt.weekday.isin([5, 6]).astype('int'),
    # df_elec1['DateHour'].dt.date는 pd.Series 입니다.
    Holiday_yn = df_elec1['DateHour'].dt.date.isin(holi).astype('int'),

```

```
Avg = df_elec1[['15분', '30분', '45분', '60분']].mean(axis=1),
TotalHour = df_elec1[['15분', '30분', '45분', '60분']].sum(axis=1),
)
```

```
In [5]: df_info1 = df_info.fillna(0)
```

```
In [6]:
```

코드는 두 개의 데이터프레임 `df\_elec1`과 `df\_info1`을 병합하여 새로운 데이터프레임 `df\_basetable1`을 생성하고, `df\_info1`의 결측치를 0으로 채웁니다.

동작 설명

1. 결측치 채우기:

- `df\_info1 = df\_info.fillna(0)`
- `df\_info` 데이터프레임의 모든 결측치(NaN)를 0으로 대체하여 새로운 데이터프레임 `df\_info1`을 생성합니다.
- `fillna(0)` 메서드를 사용하여 모든 NaN 값을 0으로 변경합니다.

2. 데이터프레임 병합:

- `df\_basetable1 = df\_elec1.merge(...)`
 - `df\_elec1`와 `df\_info1`을 특정 키(`DateHour`와 `Date`)를 기준으로 병합하여 새로운 데이터프레임 `df\_basetable1`을 생성합니다.
 - `merge` 메서드는 SQL의 JOIN 연산과 유사하게 두 데이터프레임을 합칩니다.
 - `left\_on='DateHour'`는 `df\_elec1`의 `DateHour` 열을 기준으로 병합한다는 것을 의미합니다.
 - `right\_on='Date'`는 `df\_info1`의 `Date` 열을 기준으로 병합한다는 것을 의미합니다.
 - `how='inner'`는 내부 조인(inner join) 방식을 사용하여 병합합니다.
- 내부 조인은 두 데이터프레임에서 공통적으로 존재하는 키 값에 해당하는 행만 유지합니다.

3. 불필요한 열 제거:

- `.drop(columns='Date')`
- 병합 후 `Date` 열은 더 이상 필요하지 않기 때문에 `drop` 메서드를 사용하여 제거합니다.

요약

이 코드는 다음과 같은 단계를 통해 새로운 데이터프레임을 생성합니다:

1. `df\_info`의 결측치를 0으로 채워서 `df\_info1`을 만듭니다.
2. `df\_elec1`과 `df\_info1`을 `DateHour`와 `Date` 열을 기준으로 내부 조인 방식으로 병합하여 `df\_basetable1`을 만듭니다.
3. 병합 후 불필요해진 `Date` 열을 제거하여 최종적인 `df\_basetable1` 데이터프레임을 완성합니다.

이 과정에서 `df\_elec1`의 `DateHour`와 `df\_info1`의 `Date`가 일치하는 행들만 병합되고, `df\_info1`의 결측치가 0으로 대체됩니다. 결과적으로 `df\_basetable1`은 전기 사용량 데이터와 관련 정보를 결합한 데이터프레임이 됩니다.

```
...
```

DateHour가 컬럼에 있으므로 DateHour, Date 모두 존재하여, Date는 삭제했습니다.

```
df_basetable1 = df_elec1.merge(
    df_info1,
    left_on='DateHour',
    right_on='Date', how='inner'
).drop(columns='Date')

display(df_basetable1.head())
df_basetable1.shape
```

	DateHour	15 분	30 분	45 분	60 분	DayName	Hour	AM	Weekend_yn	Holiday_yn	Avg	TotalHour	생 산 량	기 온	풍 속	습 도	강 수 량	전기요 금(계 절)	공 장 인 원	인 건 비
0	2021-01-01 00:00:00	62	61	61	61	4	0	0	0	1	61.25	245	0	-3.2	2.4	71	0.0	109.8	0.0	1.5
1	2021-01-01 01:00:00	96	93	116	113	4	1	0	0	1	104.50	418	0	-4.5	1.5	77	0.0	109.8	0.0	1.5
2	2021-01-01 02:00:00	106	96	106	107	4	2	0	0	1	103.75	415	0	-3.9	2.6	58	0.0	109.8	0.0	1.5
3	2021-01-01 03:00:00	92	110	110	109	4	3	0	0	1	105.25	421	0	-4.1	2.6	56	0.0	109.8	0.0	1.5
4	2021-01-01 04:00:00	108	105	106	108	4	4	0	0	1	106.75	427	0	-4.6	2.6	60	0.0	109.8	0.0	1.5

Out[6]: (6168, 20)

In [7]: '''

코드는 전처리 작업을 수행하여 예측 모델을 위한 데이터프레임 `df\_prob4`를 생성합니다.

동작 설명

1. 데이터프레임 정렬 및 인덱스 초기화:

- `df\_prob3 = df\_basetable1.sort\_values('DateHour').reset\_index(drop=True)`
- `df\_basetable1`을 `DateHour` 열을 기준으로 정렬합니다.
이렇게 하면 데이터가 시간 순서대로 정렬됩니다.
- `reset\_index(drop=True)` 메서드를 사용하여 기존 인덱스를 제거하고

새로운 정수형 인덱스를 할당합니다. ``drop=True``는 기존 인덱스를 새로운 열로 추가하지 않음을 의미합니다.

2. 타겟 변수 생성:

- ``df_prob3['target'] = df_prob3['TotalHour'].shift(-24)``
- ``df_prob3``에 ``target``이라는 새로운 열을 추가합니다.
- ``TotalHour`` 열의 값을 24시간 뒤로 이동시키고 ``target`` 열에 저장합니다.
예를 들어, 현재 행의 ``target`` 값은 24시간 후의 ``TotalHour`` 값이 됩니다.
이렇게 하면 24시간 후의 전력 사용량을 예측하는 문제를 만들 수 있습니다.

3. 지연 변수 생성 및 병합:

- ``df_prob4 = pd.concat([df_prob3] + [df_prob3['TotalHour'].shift(24 * i).rename('lag_{}'.format(i)) for i in range(1, 7)], axis=1).dropna()``
- ``df_prob3``와 지연 변수를 병합하여 ``df_prob4``를 생성합니다.
- ``[df_prob3]``는 기본 데이터프레임을 포함합니다.
- ``[df_prob3['TotalHour'].shift(24 * i).rename('lag_{}'.format(i)) for i in range(1, 7)]``는 ``TotalHour`` 열의 지연 변수(lagged features)를 생성합니다.
- ``shift(24 * i)``는 ``TotalHour`` 열을 24시간씩 이동시킨 값을 생성합니다.
- ``rename('lag_{}'.format(i))``는 각 지연 변수에 ``lag_1``, ``lag_2``, ..., ``lag_6``이라는 이름을 부여합니다.
- ``pd.concat(..., axis=1)``는 원본 데이터프레임 ``df_prob3``과 생성된 지연 변수들을 열(column) 방향으로 결합합니다.
- ``dropna()``는 결측값이 있는 행을 제거합니다. 지연 변수를 생성하면서 발생한 결측값을 제거하여 완전한 데이터셋을 만듭니다.

요약

- ``df_prob3``는 ``df_basetable1``을 시간 순서대로 정렬하고 인덱스를 초기화한 데이터프레임입니다. 이 데이터프레임에 ``target``이라는 새로운 열을 추가하여 24시간 후의 ``TotalHour`` 값을 예측 목표로 설정합니다.
- ``df_prob4``는 ``df_prob3``와 지연 변수(lagged features)를 결합하여 만든 데이터프레임입니다. ``TotalHour``의 1일, 2일, ..., 6일 전의 값을 각각 ``lag_1``, ``lag_2``, ..., ``lag_6``으로 추가합니다.
- 결측값이 포함된 행을 제거하여 모델 학습에 사용할 수 있는 완전한 데이터셋을 만듭니다.

이 과정은 시계열 데이터의 특성을 반영하여 예측 모델을 훈련시키기 위해 중요한 전처리 단계입니다.
...

shift를 이용합니다.

```
df_prob3 = df_basetable1.sort_values('DateHour').reset_index(drop=True)
```

```
df_prob3['target'] = df_prob3['TotalHour'].shift(-24)

df_prob4 = pd.concat(
    [df_prob3] + [df_prob3['TotalHour'].shift(24 * i).rename('lag_{}'.format(i)) for i in range(1, 7)], axis=1).dropna()

df_prob4.head()
```

Out[7]:

	DateHour	15 분	30 분	45 분	60 분	DayName	Hour	AM	Weekend_yn	Holiday_yn	...	전기 요금 (계 절)	공장인원	인 건 비	target	lag_1	lag_2	lag_3	lag_4	lag_5	lag_6
144	2021-01-07 00:00:00	22	22	22	25	3	0	0	0	0	...	109.8	0.000000	1.5	252.0	96.0	96.0	271.0	88.0	253.0	245
145	2021-01-07 01:00:00	26	22	22	22	3	1	0	0	0	...	109.8	0.369565	1.5	396.0	85.0	85.0	432.0	99.0	418.0	418
146	2021-01-07 02:00:00	22	21	21	22	3	2	0	0	0	...	109.8	0.000000	1.5	411.0	85.0	85.0	439.0	88.0	415.0	415
147	2021-01-07 03:00:00	22	22	22	23	3	3	0	0	0	...	109.8	0.629213	1.5	398.0	90.0	90.0	426.0	88.0	421.0	421
148	2021-01-07 04:00:00	22	26	26	23	3	4	0	0	0	...	109.8	2.865979	1.5	420.0	93.0	93.0	435.0	95.0	427.0	427

5 rows × 27 columns

In [8]:

```
'''
코드는 `df_basetable1` 데이터프레임을 이용하여 새로운 타겟 변수를 생성하고, 이를 원래 데이터프레임에 병합하는 작업을 수행합니다.

### 동작 설명
1. 부분 데이터프레임 생성 :
   - `df_basetable1[['DateHour', 'TotalHour']]`
   - `df_basetable1`에서 `DateHour`와 `TotalHour` 열만 선택하여 새로운 데이터프레임을 생성합니다.
'''
```


2. 날짜 이동 및 타겟 변수 생성:

- `.assign(DateHour=lambda x: x['DateHour'] - pd.Timedelta(days=1))``
- `.assign()` 메서드를 사용하여 ``DateHour`` 열의 값을 1일(24시간) 이전으로 이동시킵니다.
- ``lambda x: x['DateHour'] - pd.Timedelta(days=1)``는 ``DateHour`` 값을 하루 전으로 조정합니다.

3. 인덱스 설정 및 열 이름 변경:

- `.set_index('DateHour').rename(columns={'TotalHour': 'target'})``
- ``DateHour`` 열을 인덱스로 설정합니다.
- ``TotalHour`` 열의 이름을 ``target``으로 변경합니다. 이렇게 하면 1일 전의 ``TotalHour`` 값이 ``target`` 열에 저장됩니다.

4. 병합 (Join) 연산:

- ``df_basetable1.join(..., on='DateHour', how='inner')``
- 원래의 ``df_basetable1`` 데이터프레임과 1일 전의 ``TotalHour`` 값을 포함하는 데이터프레임을 ``DateHour`` 열을 기준으로 병합합니다
- ``on='DateHour'``는 병합 기준 열을 ``DateHour``로 설정합니다.
- ``how='inner'``는 내부 조인(inner join)을 수행하여 양쪽 데이터프레임 모두에 존재하는 ``DateHour`` 값에 대해서만 결합합니다.

전체 과정 설명

1. 원본 데이터에서 ``DateHour``와 ``TotalHour`` 열을 추출합니다.
2. ``DateHour`` 값을 하루 전으로 조정하여 새로운 데이터프레임을 생성합니다.
3. 새로운 데이터프레임의 인덱스를 ``DateHour``로 설정하고, ``TotalHour`` 열의 이름을 ``target``으로 변경합니다.
4. 원본 데이터프레임과 새로운 데이터프레임을 ``DateHour`` 열을 기준으로 내부 조인하여 병합합니다.

요약

- 이 코드는 ``df_basetable1`` 데이터프레임에서 각 행의 ``DateHour`` 값을 하루 전으로 조정한 후, 그 시점의 ``TotalHour`` 값을 ``target``이라는 새로운 열로 추가합니다.
- 결과적으로, 원래 데이터프레임에 하루 전의 전력 사용량(``TotalHour``)이 ``target`` 열로 추가됩니다.
- 이 과정은 시계열 데이터 분석에서 주로 사용되는 방법으로, 예를 들어 특정 시점의 데이터와 이전 시점의 데이터를 비교하거나, 이전 시점의 데이터를 예측 모델의 입력으로 사용하는 경우에 유용합니다.

...

shift를 사용하지 않고 target를 구하는 방법입니다.# *(값을 구하기만 하고, 따로 prob4로 만들지는 않았습니다.)*

```
df_basetable1.join(
    df_basetable1[['DateHour', 'TotalHour']].assign(
        DateHour=lambda x: x['DateHour'] - pd.Timedelta(days=1)
    ).set_index('DateHour').rename(columns={'TotalHour': 'target'}),
    on='DateHour', how='inner'
)
```

Out[8]:

	DateHour	15 분	30 분	45 분	60 분	DayName	Hour	AM	Weekend_yn	Holiday_yn	...	TotalHour	생산 량	기 온	풍 속	습 도	강 수 량	전기 요금 (계 절)	공장인원	인 건 비	targ	
0	2021-01-01 00:00:00	62	61	61	61		4	0	0		0	1 ...	245	0	-3.2	2.4	71	0.0	109.8	0.000000	1.5	2
1	2021-01-01 01:00:00	96	93	116	113		4	1	0		0	1 ...	418	0	-4.5	1.5	77	0.0	109.8	0.000000	1.5	4
2	2021-01-01 02:00:00	106	96	106	107		4	2	0		0	1 ...	415	0	-3.9	2.6	58	0.0	109.8	0.000000	1.5	4
3	2021-01-01 03:00:00	92	110	110	109		4	3	0		0	1 ...	421	0	-4.1	2.6	56	0.0	109.8	0.000000	1.5	4
4	2021-01-01 04:00:00	108	105	106	108		4	4	0		0	1 ...	427	0	-4.6	2.6	60	0.0	109.8	0.000000	1.5	4
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
6139	2021-09-13 19:00:00	162	160	148	122		0	19	1		0	0 ...	592	2820	22.3	2.1	78	0.0	167.2	4.763514	1.5	6
6140	2021-09-13 20:00:00	113	122	122	126		0	20	1		0	0 ...	483	14	22.2	1.3	78	0.0	167.2	0.028986	1.5	5
6141	2021-09-13 21:00:00	122	131	122	115		0	21	1		0	0 ...	490	72	21.9	1.9	77	0.0	167.2	0.146939	1.5	5
6142	2021-09-13 22:00:00	97	108	122	113		0	22	1		0	0 ...	440	11	21.7	1.4	77	0.0	167.2	0.025000	1.5	4
6143	2021-09-13 23:00:00	111	123	110	87		0	23	1		0	0 ...	431	345	21.4	1.4	80	0.0	167.2	0.800464	1.5	4

6144 rows × 21 columns

In [9]:

```
'''
코드는 주어진 데이터프레임 `df_prob4`를 사용하여 학습용 데이터와 테스트용 데이터를 나누고,
테스트 데이터에서 타겟 변수(`target`)를 분리하는 작업을 수행합니다.
이 작업은 일반적으로 머신러닝 모델을 학습하고 평가하기 위해 수행됩니다.

### 동작 설명
1. 데이터프레임 분할:
- `df_prob4` 데이터프레임을 `DateHour` 열을 기준으로 학습용(`df_prob6_train`)과 테스트용(`df_prob6_test`) 데이터로 분할합니다.
- `df_prob4['DateHour'] < pd.to_datetime('2021-08-14')` 조건을 만족하는 행들을 `df_prob6_train`에 저장합니다.
- `df_prob4['DateHour'] >= pd.to_datetime('2021-08-14')` 조건을 만족하는 행들을 `df_prob6_test`에 저장합니다.

2. 타겟 변수 분리:
- `df_prob6_test` 데이터프레임에서 `target` 열을 제거하고, 그 값을 `s_kaggle_ans` 시리즈에 저장합니다.
- `pop` 메서드는 지정한 열을 데이터프레임에서 제거하면서 그 값을 반환합니다.
- 이렇게 분리된 `target` 값은 머신러닝 모델의 예측 성능을 평가하기 위해 사용될 수 있습니다.
    여기서는 Kaggle의 정답으로 간주하겠다고 언급되어 있습니다.

### 전체 과정 설명
1. 데이터 분할:
- `df_prob4` 데이터프레임을 날짜(`DateHour`)를 기준으로 두 개의 데이터프레임(`df_prob6_train`과 `df_prob6_test`)으로 분할합니다.
- `2021-08-14` 이전의 데이터를 학습용 데이터로, `2021-08-14` 이후의 데이터를 테스트용 데이터로 사용합니다.

2. 타겟 변수 분리:
- 테스트용 데이터프레임 `df_prob6_test`에서 타겟 변수(`target`)를 제거하고, 이를 `s_kaggle_ans`에 저장합니다.
- `s_kaggle_ans`는 모델의 예측 결과를 평가하는 데 사용될 것입니다.

### 요약
- 이 코드는 시계열 데이터를 학습용과 테스트용으로 분할하고, 테스트 데이터에서 타겟 변수를 분리하는 작업을 수행합니다.
- 분리된 타겟 변수는 모델의 예측 성능을 평가하는 데 사용됩니다.
- 이 과정은 머신러닝 모델의 학습과 평가에서 매우 일반적으로 사용되는 절차입니다.
'''

# 문제에서 제시한 데이터셋을 만듭니다.
df_prob6_train = df_prob4.loc[df_prob4['DateHour'] < pd.to_datetime('2021-08-14')]
df_prob6_test = df_prob4.loc[df_prob4['DateHour'] >= pd.to_datetime('2021-08-14')]
# 정답 target은 떼어냅니다.

# Kaggle의 정답으로 간주하겠습니다, 실제 시험에서는 공개 되지 않습니다.
```

```
s_kaggle_ans = df_prob6_test.pop('target')
s_kaggle_ans
```

```
Out[9]: 5400      86.0
        5401      92.0
        5402      87.0
        5403      85.0
        5404      88.0
        ...
        6139     613.0
        6140     512.0
        6141     513.0
        6142     443.0
        6143     439.0
Name: target, Length: 744, dtype: float64
```

Kaggle형 풀이 단계

Step 1: 검증 방법을 정하고, 검증 루틴을 만듭니다.

Step 2: Baseline 모델을 만듭니다

Step 3: 모델 선택 루틴을 만듭니다.

Step 4: 모델 개선 작업을 합니다.

Step1: 검증 방법을 정하고, 검증 루틴을 만듭니다.

```
In [10]: df_train = df_prob6_train.loc[df_prob6_train['DateHour'] < pd.to_datetime('2021-07-14')]
df_valid = df_prob6_train.loc[df_prob6_train['DateHour'] >= pd.to_datetime('2021-07-14')]
```

```
In [11]: # 모델의 검증 결과를 남겨두기 위한 리스트입니다.
s_hist = []
```

```
In [12]: '''
코드는 주어진 데이터를 학습용과 검증용 데이터로 분할하고, 여러 머신러닝 모델의 성능을 평가한 후,
```

최적의 모델을 선택하여 예측 결과를 저장하는 작업을 수행합니다.

동작 설명

1. 데이터 분할:

- ``df_prob6_train`` 데이터프레임을 ``DateHour`` 열을 기준으로 다시 학습용(``df_train``)과 검증용(``df_valid``) 데이터로 분할합니다.
- ``2021-07-14`` 이전의 데이터를 학습용 데이터로, ``2021-07-14`` 이후의 데이터를 검증용 데이터로 사용합니다.

2. 검증 결과 출력 및 기록 함수 (``eval_model``):

- ``eval_model`` 함수는 주어진 모델(``reg_model``)을 학습용 데이터(``df_train``)로 학습시킨 후, 검증용 데이터(``df_valid``)와 학습용 데이터 모두에 대해 예측을 수행합니다.
- 검증 데이터와 학습 데이터에 대한 평균 절대 오차(``mean_absolute_error``)를 계산하여 문자열 ``s``로 저장합니다.
- 결과를 ``s_hist`` 리스트에 기록하고, 마지막으로 각 모델의 결과를 출력합니다.

3. 모델 선택 및 예측 함수 (``select_model``):

- ``select_model`` 함수는 주어진 모델(``reg_model``)을 전체 학습 데이터(``df_prob6_train``)로 학습시킨 후, 테스트 데이터(``df_prob6_test``)에 대해 예측을 수행합니다.
- 예측 결과는 ``DateHour``와 ``TotalHour`` 열을 가진 데이터프레임으로 만들어 CSV 파일(``answer6.csv``)로 저장합니다.
- 예측된 결과(``prd``)를 반환합니다.

4. 특징 변수 리스트 (``X_all``):

- ``X_all`` 변수는 테스트 데이터(``df_prob6_test``)의 모든 열 이름을 리스트로 저장합니다.
- 이 리스트는 모델 학습 및 예측 시 사용될 특징 변수(피처)들을 지정합니다.

전체 과정 설명

1. 데이터를 학습용과 검증용으로 분할하여, 학습 모델의 성능을 평가하기 위한 별도의 검증 데이터를 확보합니다.
2. ``eval_model`` 함수를 사용하여 다양한 모델의 성능을 평가하고, 그 결과를 기록하고 출력합니다.
3. ``select_model`` 함수를 사용하여 최종 모델을 선택하고, 테스트 데이터에 대해 예측을 수행하여 결과를 저장합니다.
4. ``X_all`` 변수에 모든 특징 변수를 저장하여, 모델 학습과 예측에 사용할 피처들을 지정합니다.

코드는 모델 학습과 성능 평가, 최종 모델 선택 및 예측 결과 저장까지의 전체 과정을 체계적으로 수행합니다.

```
...
```

```
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVR
from sklearn.metrics import mean_absolute_error
```

```
# 검증 결과 출력 루틴을 만듭니다. 이와 함께 결과를 기록합니다.
def eval_model(model_name, reg_model):
    reg_model.fit(df_train[X_all], df_train['target'])
```

```
s = "{:.5f}, {:.5f}".format(
    mean_absolute_error(df_valid['target'], reg_model.predict(df_valid[X_all])),
    mean_absolute_error(df_train['target'], reg_model.predict(df_train[X_all]))
)
print(s)

s_hist.append(pd.Series([model_name, s], index=['model_name', 'result']))
display(pd.DataFrame(s_hist).groupby('model_name').last())

def select_model(reg_model):
    reg_model.fit(df_prob6_train[X_all], df_prob6_train['target'])
    prd = reg_model.predict(df_prob6_test[X_all])
    pd.DataFrame({
        'DateHour': df_prob6_test['DateHour'],
        'TotalHour': prd
    }).set_index('DateHour').to_csv('answer6.csv')
    return prd

X_all = df_prob6_test.columns.tolist()
X_all
```

```
Out[12]: ['DateHour',
          '15분 ',
          '30분 ',
          '45분 ',
          '60분 ',
          'DayName',
          'Hour',
          'AM',
          'Weekend_yn',
          'Holiday_yn',
          'Avg',
          'TotalHour',
          '생산량',
          '기온',
          '풍속',
          '습도',
          '강수량',
          '전기요금(계절)',
          '공장인원',
          '인건비',
          'lag_1',
          'lag_2',
          'lag_3',
          'lag_4',
          'lag_5',
          'lag_6']
```

```
In [13]: #위 2개 함수 추가설명
```

```
...
```

```
두 개의 함수 `eval_model`과 `select_model`은 회귀 모델의 성능을 평가하고, 최종 모델을 선택하여 예측 결과를 저장하는 역할을 합니다.
```

```
...
```

```
# `eval_model` 함수
```

```
...
```

```
이 함수는 주어진 회귀 모델을 학습시키고, 모델의 성능을 평가하여 결과를 출력하고 저장합니다.
```

```
def eval_model(model_name, reg_model):
```

```
    # 모델을 학습 데이터(df_train)로 학습시킵니다.
```

```
    reg_model.fit(df_train[X_all], df_train['target'])
```

```
    # 검증 데이터(df_valid)와 학습 데이터(df_train)에 대한 예측을 수행하고,mean_absolute_error (MAE)를 계산합니다.
```

```
    s = "{:.5f}, {:.5f}".format(
```

```

        mean_absolute_error(df_valid['target'], reg_model.predict(df_valid[X_all])),
        mean_absolute_error(df_train['target'], reg_model.predict(df_train[X_all]))
    )

    # 계산된 MAE를 포맷팅하여 출력합니다.
    print(s)

    # 결과를 저장하기 위해 시리즈를 생성하고, 이를 s_hist 리스트에 추가합니다.
    s_hist.append(pd.Series([model_name, s], index=['model_name', 'result']))

    # 모델 이름별로 마지막 결과를 그룹화하여 표시합니다.
    display(pd.DataFrame(s_hist).groupby('model_name').last())

## 주요 단계
1. 모델 학습: `reg_model.fit(df_train[X_all], df_train['target'])`
    - 주어진 모델을 학습 데이터 `df_train`의 독립 변수 `X_all`과 종속 변수 `target`으로 학습시킵니다.
2. 예측 및 평가:
    - `mean_absolute_error` 함수를 사용하여 검증 데이터 `df_valid`와 학습 데이터 `df_train`의 예측 오차를 계산합니다.
    - `s` 변수에 두 MAE 값을 포맷팅하여 저장합니다.
3. 결과 출력: `print(s)`
    - 포맷팅된 MAE 값을 출력합니다.
4. 결과 저장 및 표시:
    - `s_hist` 리스트에 모델 이름과 결과를 포함하는 시리즈를 추가합니다.
    - `display` 함수를 사용하여 `s_hist`를 데이터프레임으로 변환하고, 모델 이름별로 마지막 결과를 그룹화하여 표시합니다.
...

# `select_model` 함수
...

이 함수는 주어진 회귀 모델을 사용하여 최종 예측을 수행하고, 예측 결과를 CSV 파일로 저장합니다.

def select_model(reg_model):
    # 모델을 학습 데이터(df_prob6_train)로 학습시킵니다.
    reg_model.fit(df_prob6_train[X_all], df_prob6_train['target'])

    # 테스트 데이터(df_prob6_test)에 대한 예측을 수행합니다.
    prd = reg_model.predict(df_prob6_test[X_all])

    # 예측 결과를 데이터프레임으로 생성하고, 'answer6.csv' 파일로 저장합니다.
    pd.DataFrame({
        'DateHour': df_prob6_test['DateHour'],
        'TotalHour': prd
    }).set_index('DateHour').to_csv('answer6.csv')

```



```

    # 예측 결과를 반환합니다.
    return prd

## 주요 단계
1. 모델 학습: `reg_model.fit(df_prob6_train[X_all], df_prob6_train['target'])`
    - 주어진 모델을 학습 데이터 `df_prob6_train`의 독립 변수 `X_all`과 종속 변수 `target`으로 학습시킵니다.
2. 예측 수행: `prd = reg_model.predict(df_prob6_test[X_all])`
    - 학습된 모델을 사용하여 테스트 데이터 `df_prob6_test`에 대한 예측을 수행하고, 예측값을 `prd`에 저장합니다.
3. 결과 저장:
    - 예측 결과를 데이터프레임으로 생성하고, 'answer6.csv' 파일로 저장합니다.
      데이터프레임에는 `DateHour` 열을 인덱스로 설정하여 저장합니다.
4. 예측 결과 반환: `return prd`
    - 예측값 배열 `prd`를 반환합니다.
...

## 요약
...

- `eval_model` 함수는 모델을 학습시키고, 학습 데이터와 검증 데이터에 대한 MAE를 계산하여 출력하고, 결과를 저장합니다.
- `select_model` 함수는 모델을 학습시키고, 테스트 데이터에 대한 예측을 수행하여 예측 결과를 CSV 파일로 저장합니다.
  또한 예측값을 반환합니다.

이 두 함수는 모델 평가 및 선택 과정에서 중요한 역할을 하며,
`eval_model`은 모델 성능 평가에, `select_model`은 최종 예측값 저장에 사용됩니다.
...

print('')
```

Step2: Baseline 만들기

```

In [14]: ...

`ColumnTransformer`는 Scikit-Learn 라이브러리에서 제공하는 도구로, 여러 가지 전처리 변환을
데이터의 서로 다른 열에 각각 적용할 수 있도록 해줍니다.
이를 통해 데이터 전처리 과정을 단순화하고 체계적으로 관리할 수 있습니다.

### 주요 인자
1. transformers (필수):
    - 리스트 형식으로, 각 요소는 변환기에 대한 정보를 포함합니다.
    - 각 요소는 `(name, transformer, columns)` 형태의 튜플로 구성됩니다.
```

- `name`: 변환기의 이름으로, 문자열로 지정합니다.
- `transformer`: 적용할 변환기 객체입니다.
 Scikit-Learn의 변환기(`StandardScaler`, `OneHotEncoder`, `PCA` 등)나 `'passthrough'`(열을 그대로 통과시키는 경우) 또는 `'drop'`(열을 삭제하는 경우)을 지정할 수 있습니다.
- `columns`: 변환기를 적용할 열 이름이나 열의 인덱스입니다. 리스트, 슬라이스, 배열, 또는 불리언 마스크 등이 가능합니다.

2. `remainder` (옵션):

- `'drop'` 또는 `'passthrough'` 값을 가지며, 기본값은 `'drop'`입니다.
- 지정되지 않은 나머지 열을 어떻게 처리할지를 결정합니다.
 - `'drop'`: 나머지 열을 삭제합니다.
 - `'passthrough'`: 나머지 열을 변환하지 않고 그대로 통과시킵니다.

3. `sparse_threshold` (옵션):

- 희소 행렬로 변환할 임계값을 설정합니다.
- 기본값은 `0.3`으로, 변환된 결과의 희소성 비율이 이 임계값보다 크면 희소 행렬로 변환됩니다.

4. `n_jobs` (옵션):

- 병렬로 작업을 실행할 때 사용할 CPU 코어 수를 설정합니다. `-1`로 설정하면 모든 가용 코어를 사용합니다.
- 기본값은 `None`으로, 직렬로 실행됩니다.

5. `transformer_weights` (옵션):

- 변환기의 가중치를 설정합니다. 변환기의 이름을 키로 하고 가중치 값을 값으로 하는 딕셔너리를 사용합니다.
- 기본값은 `None`입니다.

6. `verbose` (옵션):

- `True`로 설정하면 변환기 정보를 출력합니다.
- 기본값은 `False`입니다.

반환값

- `ColumnTransformer` 객체: 지정된 변환을 수행할 수 있는 객체를 반환합니다.
 이 객체는 Scikit-Learn의 다른 변환기 및 추정기와 마찬가지로 `'fit'`, `'transform'`, `'fit_transform'` 메서드를 가지고 있습니다.

예제

아래 예제는 여러 가지 변환을 각기 다른 열에 적용하는 방법을 보여줍니다.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.svm import SVR
```

```
# 예제 데이터프레임
import pandas as pd
```

```

df = pd.DataFrame({
    'DayName': ['Mon', 'Tue', 'Wed', 'Thu', 'Fri'],
    'TotalHour': [10, 20, 15, 10, 5],
    'AM': [1, 0, 1, 0, 1],
    'lag_1': [9, 19, 14, 11, 6],
    'lag_2': [8, 18, 13, 12, 7]
})

# ColumnTransformer를 정의합니다.
ct = ColumnTransformer([
    ('ohe', OneHotEncoder(categories='auto', drop='first'), ['DayName']),
    ('std', StandardScaler(), ['lag_1', 'lag_2']),
    ('pt', 'passthrough', ['AM'])
])

# 변환 파이프라인을 구성합니다.
pipeline = Pipeline([
    ('transformer', ct),
    ('regressor', SVR(C=10, kernel='rbf', gamma=0.1))
])

# 데이터를 변환하고 학습합니다.
X = df.drop(columns='TotalHour') # 'TotalHour' 열 제외
y = df['TotalHour']
pipeline.fit(X, y)

# 예측을 수행합니다.
predictions = pipeline.predict(X)
print(predictions)

# 결과
[ 9.90025269 19.89955562 14.90025206 10.09993895  7.04346851]

### 동작 과정
1. `ColumnTransformer` 정의:
   - `OneHotEncoder`를 `DayName` 열에 적용하여 범주형 변수를 이진 벡터로 변환합니다.
   - `StandardScaler`를 `TotalHour`, `lag_1`, `lag_2` 열에 적용하여 평균이 0, 표준편차가 1이 되도록 정규화합니다.
   - `AM` 열은 변환 없이 그대로 사용합니다(`passthrough`).

2. 파이프라인 생성:
   - `ColumnTransformer`와 `SVR` 회귀 모델을 결합하여 파이프라인을 생성합니다.

```

3. 모델 학습:

- 주어진 데이터 `X`와 타겟 `y`를 사용하여 파이프라인을 학습시킵니다.

4. 예측 수행:

- 학습된 모델을 사용하여 예측을 수행합니다.

이렇게 `ColumnTransformer`를 사용하면 다양한 전처리 작업을 간편하게 수행하고,
이를 하나의 파이프라인으로 결합하여 일관된 데이터 처리와 모델 학습을 할 수 있습니다.

```
...
```

```
...
```

코드는 특성 변환과 지원 벡터 회귀(SVR) 모델을 결합하여 학습 데이터와 검증 데이터를 사용해 모델의 성능을 평가하는 작업을 수행합니다.
주요 구성 요소와 전체 동작을 자세히 설명하겠습니다.

주요 구성 요소

1. `ColumnTransformer` (ct):

```
ct = ColumnTransformer([
    ('ohe', OneHotEncoder(categories='auto', drop='first'), ['DayName']),
    ('std', StandardScaler(), ['TotalHour'] + ['lag_{}'.format(i)
                                                for i in range(1, 7)]),
    ('pt', 'passthrough', ['AM'])
])
```

- `ColumnTransformer`는 여러 가지 전처리 변환을 각기 다른 열에 적용할 수 있게 해주는 도구입니다.
- One-Hot Encoding (ohe):
 - `DayName` 열에 대해 `OneHotEncoder`를 적용하여 범주형 변수를 이진 벡터로 변환합니다.
 - `drop='first'` 옵션을 사용하여 첫 번째 범주를 제거함으로써 다중공선성을 줄입니다.
- Standard Scaling (std):
 - `TotalHour`와 `lag\_1`부터 `lag\_6`까지의 열에 대해 `StandardScaler`를 적용하여 평균이 0, 표준편차가 1이 되도록 정규화합니다.
- Passthrough (pt):
 - `AM` 열은 그대로 통과시킵니다(passthrough).

2. `make\_pipeline`:

```
reg_svr = make_pipeline(ct, SVR(C=10, kernel='rbf', gamma=0.1))
```

- `make\_pipeline`은 변환기(`ct`)와 추정기(`SVR`)를 결합하여 하나의 파이프라인으로 만듭니다.
- SVR 모델:
 - `SVR(C=10, kernel='rbf', gamma=0.1)`는 커널로 RBF(방사 기저 함수)를 사용하고, 정규화 매개변수 `C`는 10, `gamma`는 0.1로 설정한 지원 벡터 회귀 모델입니다.
 - `SVR` 모델은 주어진 특성들을 사용하여 회귀 분석을 수행합니다.

3. `eval\_model` 함수:

```
eval_model('baseline', reg_svr)
```

- ``eval_model`` 함수는 이전에 정의된 함수로, 주어진 모델(``reg_model``)을 학습용 데이터(``df_train``)로 학습시킨 후, 검증용 데이터(``df_valid``)와 학습용 데이터 모두에 대해 예측을 수행합니다.
- 평균 절대 오차(``mean_absolute_error``)를 계산하여 결과를 출력하고 기록합니다.

전체 동작 과정

1. 특성 변환 정의:

- ``ColumnTransformer``를 사용하여 각 열에 대한 적절한 전처리 변환을 정의합니다.
이는 데이터가 학습 및 예측 단계에서 일관되게 처리될 수 있도록 합니다.
- 범주형 변수인 ``DayName``은 **One-Hot Encoding**을 통해 변환되고, 수치형 변수인 ``TotalHour``와 ``lag_1``부터 ``lag_6``은 **Standard Scaling**을 통해 정규화됩니다. ``AM`` 변수는 변환 없이 그대로 사용됩니다.

2. 파이프라인 생성:

- ``make_pipeline``을 사용하여 특성 변환기(``ct``)와 지원 벡터 회귀 모델(``SVR``)을 하나의 파이프라인으로 결합합니다.
이는 데이터가 파이프라인을 통과하면서 자동으로 변환되고 모델에 입력되도록 합니다.

3. 모델 평가:

- ``eval_model`` 함수를 사용하여 생성된 파이프라인(``reg_svr``)을 학습용 데이터로 학습시키고, 검증용 데이터와 학습용 데이터에 대한 예측 결과를 평가합니다.
- 평균 절대 오차를 계산하여 모델의 성능을 평가하고, 결과를 출력하고 기록합니다.

이 과정을 통해 데이터 전처리와 모델 학습을 일관되게 수행할 수 있으며, 다양한 모델과 변환기를 손쉽게 테스트하고 비교할 수 있습니다.

```
ct = ColumnTransformer([
    ('ohe', OneHotEncoder(categories='auto', drop='first'), ['DayName']),
    ('std', StandardScaler(), ['TotalHour'] + ['lag_{}'.format(i) for i in range(1, 7)]),
    ('pt', 'passthrough', ['AM'])
])

reg_svr = make_pipeline(ct, SVR(C=10, kernel='rbf', gamma=0.1))
eval_model('baseline', reg_svr)
```

146.31594, 59.58739

result

model_name

baseline 146.31594, 59.58739

In [37]: # 위 코드에 대한 추가 설명

...

``Pipeline()``과 ``make_pipeline()``은 모두 `scikit-learn`에서 제공하는 기능으로, 두 방법 모두 파이프라인을 구성하는 데 사용됩니다. 그러나 사용 방식과 특정 상황에서의 유용성에 약간의 차이가 있습니다.

Pipeline() 사용

1. 직접적인 제어:

- ``Pipeline`` 객체를 생성할 때는 각 단계에 대해 이름을 명시적으로 지정해야 합니다.
- 예를 들어, ``('transformer', ct)``와 ``('regressor', SVR(C=10, kernel='rbf', gamma=0.1))``처럼 단계별로 명명합니다.
- 명시적으로 이름을 지정하기 때문에 각 단계에 대한 접근과 디버깅이 용이합니다.

2. 명시적 구성:

- ``Pipeline``을 사용할 때는 파이프라인의 각 단계에 이름을 부여해야 하므로 코드가 더 명확해집니다.
- 이는 특히 여러 단계를 포함하는 복잡한 파이프라인에서 유용합니다.

3. 예시:

```
ct = ColumnTransformer([
    ('ohe', OneHotEncoder(categories='auto', drop='first'), ['DayName']),
    ('std', StandardScaler(), ['lag_1', 'lag_2']),
    ('pt', 'passthrough', ['AM'])
])

pipeline = Pipeline([
    ('transformer', ct),
    ('regressor', SVR(C=10, kernel='rbf', gamma=0.1))
])
```

make_pipeline() 사용

1. 간결한 구성:

- ``make_pipeline``은 단계를 자동으로 이름 지정하여, 코드가 간결해집니다.
- 각 단계의 이름은 해당 단계의 클래스 이름을 소문자로 변환한 값으로 자동 지정됩니다.

2. 간편한 사용:

- 코드가 간결해지며, 간단한 파이프라인을 빠르게 작성할 때 유용합니다.
- 그러나, 각 단계에 명시적 이름을 부여하지 않기 때문에 특정 단계에 접근하거나 디버깅할 때는 불편할 수 있습니다.

3. 예시:

```
ct = ColumnTransformer([
    ('ohe', OneHotEncoder(categories='auto', drop='first'), ['DayName']),
    ('std', StandardScaler(), ['TotalHour'] + ['lag_{}'.format(i) for i in range(1, 7)]),
    ('pt', 'passthrough', ['AM'])
])
```

```
reg_svr = make_pipeline(ct, SVR(C=10, kernel='rbf', gamma=0.1))
```

차이점 요약

1. 명명 규칙:
 - `Pipeline`에서는 각 단계의 이름을 명시적으로 지정할 수 있어 접근성과 디버깅이 용이합니다.
 - `make\_pipeline`은 단계를 자동으로 명명하므로 코드가 간결해집니다.
2. 유연성 및 명확성:
 - `Pipeline`은 명확성과 유연성이 높아 복잡한 파이프라인을 다룰 때 유리합니다.
 - `make\_pipeline`은 간단한 파이프라인을 빠르게 작성할 때 유리합니다.
3. 디버깅:
 - `Pipeline`을 사용하면 명시적 이름을 통해 각 단계에 접근하기 쉬워 디버깅이 수월합니다.
 - `make\_pipeline`은 디버깅 시 단계의 자동 명칭 때문에 약간의 불편함이 있을 수 있습니다.

결론

- 복잡한 파이프라인: `Pipeline`을 사용하여 각 단계에 명확한 이름을 부여하고 접근성을 높이는 것이 좋습니다.
- 간단한 파이프라인: `make\_pipeline`을 사용하여 간편하고 빠르게 파이프라인을 구성하는 것이 좋습니다.

```
'''
print('')
```

Step 3: 모델 선택 루틴을 만듭니다.

```
In [16]: '''
이 코드는 SVR(Support Vector Regression) 모델을 사용하여 예측을 수행하고, 그 결과를 출력하는 부분입니다.
1. `select_model(reg_svr)`: 이 함수는 SVR 모델을 훈련 데이터에 적합하게 합니다.
   그런 다음 테스트 데이터에 대해 예측을 수행하고 결과를 반환합니다. 이 결과는 `prd` 변수에 저장됩니다.
2. `print('자가채점:{:}'.format(mean_absolute_error(s_kaggle_ans, prd)))`:
   이 부분은 예측 결과와 실제 정답인 `s_kaggle_ans` 간의 평균 절대 오차를 계산하고, 그 값을 출력합니다.
   이 값은 모델의 성능을 나타내는데, 평균 절대 오차가 작을수록 모델의 예측이 정확하다는 것을 의미합니다.

즉, 이 코드는 SVR 모델을 사용하여 테스트 데이터에 대한 예측을 수행하고, 그 결과의 성능을 평가하는 역할을 합니다.
'''

prd = select_model(reg_svr)
print('자가채점:{:}'.format(mean_absolute_error(s_kaggle_ans, prd)))
```

자가채점 :28.966073501836416

Step 4: 모델 개선 작업을 합니다.

In [17]:

```
'''
코드는 다음과 같은 작업을 수행합니다.

1. `ColumnTransformer`를 사용하여 데이터를 전처리합니다. 이 때 세 가지 변환을 수행합니다.
  - 'DayName' 열에 대해 One-Hot 인코딩을 수행합니다. 이를 통해 범주형 변수를 숫자형으로 변환합니다.
  - 'TotalHour' 및 'lag_1'부터 'lag_6'까지의 열에 대해 표준화(StandardScaler)를 수행합니다.
    이는 각 열의 평균을 0으로, 표준 편차를 1로 만들어 줍니다.
  - 'AM' 열에 대해서는 추가적인 변환 없이 그대로 사용합니다.

2. 이렇게 전처리된 데이터를 SVR(Support Vector Regression) 모델에 입력하여 학습합니다.
  SVR의 파라미터로는 C, kernel, gamma 등이 설정되어 있습니다.

3. `eval_model` 함수를 호출하여 모델의 성능을 평가합니다. 이 함수는 모델을 학습하고 검증 데이터에 대한 예측을 수행한 후,
  평균 절대 오차(Mean Absolute Error)를 출력합니다. 이 값은 모델의 예측 성능을 나타내는 지표 중 하나입니다.

따라서 이 코드는 전처리된 데이터를 사용하여 SVR 모델을 학습하고, 이 모델의 성능을 평가하는 과정을 수행합니다.
'''

ct = ColumnTransformer([
    ('ohe', OneHotEncoder(categories='auto', drop='first'), ['DayName']),
    ('std', StandardScaler(), ['TotalHour'] + ['lag_{}'.format(i) for i in range(1, 7)]),
    ('pt', 'passthrough', ['AM'])
])

reg_svr2 = make_pipeline(ct, SVR(C=1000, kernel='rbf', gamma=0.03))
eval_model('svr2', reg_svr2)
```

126.06277, 47.94924

result**model_name**

baseline 146.31594, 59.58739

svr2 126.06277, 47.94924


```
In [18]: '''
코드는 다음과 같은 작업을 수행합니다.

1. `select_model` 함수를 사용하여 주어진 SVR 모델(`reg_svr2`)을 학습하고, 테스트 데이터에 대한 예측값을 생성합니다.
   이 함수는 학습된 모델을 사용하여 테스트 데이터에 대한 예측값을 생성하고, 이를 CSV 파일로 저장한 뒤, 예측값을 반환합니다.

2. 반환된 예측값과 실제 정답(`s_kaggle_ans`)을 사용하여 평균 절대 오차(Mean Absolute Error)를 계산합니다.
   이 값은 모델의 예측 성능을 나타내는 지표 중 하나이며, 예측값과 실제값 간의 평균적인 절대적인 차이를 나타냅니다.

따라서 이 코드는 주어진 SVR 모델을 사용하여 테스트 데이터에 대한 예측값을 생성하고, 이를 사용하여 모델의 성능을 평가합니다.
'''

prd = select_model(reg_svr2)
print('자가채점:{}'.format(mean_absolute_error(s_kaggle_ans, prd)))
```

자가채점:29.808029150837296

타당한 선택이라 판단됩니다.

실제로는 안 좋습니다. 데이터셋이 변동성이 심한 특징이 있습니다.

이를 감안하여 채점 기준을 만들 것입니다.

```
In [19]: '''
`SVR` (Support Vector Regression)은 scikit-learn 라이브러리에서 제공하는 회귀 모델로,
서포트 벡터 머신(SVM)을 사용하여 회귀 분석을 수행합니다.
`SVR`은 회귀 문제에서 데이터 포인트가 목표값에서  $\epsilon$ -허용 오차 내에 있도록 하고,
허용 오차를 벗어난 경우 페널티를 부여하여 모델을 학습시킵니다.

### 인자 (Parameters)
`SVR`의 주요 인자는 다음과 같습니다:

1. kernel:
   - 설명: 사용할 커널 함수의 유형을 지정합니다. 커널 함수는 비선형 데이터의 변환을 통해 선형 분리 가능하게 만드는 함수입니다.
   - 타입: str, default='rbf'
   - 선택값: 'linear', 'poly', 'rbf', 'sigmoid', 'precomputed'
   - 예시: kernel='linear'

2. degree:
   - 설명: 다항 커널(poly kernel)을 사용할 때 다항식의 차수를 지정합니다.
   - 타입: int, default=3
   - 예시: degree=3
```

3. **gamma**:
- 설명: 'rbf', 'poly', 'sigmoid' 커널에 사용되는 커널 계수.
 - 타입: {'scale', 'auto'} or float, default='scale'
 - 기본값 설명:
 - 'scale': $1 / (n_features * X.var())$ (기본값)
 - 'auto': $1 / n_features$
 - 예시: gamma='scale'
4. **coef0**:
- 설명: 다항 및 시그모이드 커널 함수에서 독립 항을 지정합니다.
 - 타입: float, default=0.0
 - 예시: coef0=0.0
5. **tol**:
- 설명: 허용 오차를 지정합니다.
 - 타입: float, default=1e-3
 - 예시: tol=1e-3
6. **C**:
- 설명: 규제 매개변수. 작을수록 모델이 더 규제가 많아지고, 클수록 덜 규제가 됩니다.
 - 타입: float, default=1.0
 - 예시: C=1.0
7. **epsilon**:
- 설명: ϵ -튜브의 폭. 이 값 내에서 오류가 무시됩니다.
 - 타입: float, default=0.1
 - 예시: epsilon=0.1
8. **shrinking**:
- 설명: 축소 휴리스틱을 사용할지 여부를 지정합니다.
 - 타입: bool, default=True
 - 예시: shrinking=True
9. **cache_size**:
- 설명: 커널 캐시의 크기를 메가바이트 단위로 지정합니다.
 - 타입: float, default=200
 - 예시: cache_size=200
10. **verbose**:
- 설명: 상세 출력을 활성화합니다.
 - 타입: bool, default=False
 - 예시: verbose=False
11. **max_iter**:
- 설명: 알고리즘의 최대 반복 횟수. -1이면 무제한 반복합니다.
 - 타입: int, default=-1
 - 예시: max_iter=-1

리턴값 (Returns)

`SVR` 객체 자체는 별도의 리턴값을 제공하지 않지만, 이 클래스를 통해 생성된 객체는 여러 메서드를 통해 다양한 기능을 제공합니다. 주요 메서드는 다음과 같습니다:

```
1. fit(X, y):
    - 설명: 모델을 학습시킵니다.
    - 인자:
        - `X`: 훈련 데이터 (특징 행렬)
        - `y`: 타겟 값 (레이블)
    - 리턴값: 학습된 `SVR` 객체
    - 예시: svr.fit(X_train, y_train)

2. predict(X):
    - 설명: 입력 데이터에 대한 예측값을 반환합니다.
    - 인자:
        - `X`: 예측할 데이터 (특징 행렬)
    - 리턴값: 예측값의 배열
    - 예시: predictions = svr.predict(X_test)

3. score(X, y):
    - 설명: 모델의 성능을 평가합니다.
    - 인자:
        - `X`: 테스트 데이터 (특징 행렬)
        - `y`: 실제 값 (레이블)
    - 리턴값: 성능 점수 ( $R^2$  score)
    - 예시: score = svr.score(X_test, y_test)

4. get_params(deep=True):
    - 설명: 모델의 매개변수를 딕셔너리 형태로 반환합니다.
    - 인자:
        - `deep`: 매개변수 딕셔너리에 포함된 객체의 매개변수도 반환할지 여부
    - 리턴값: 매개변수 딕셔너리
    - 예시: params = svr.get_params()

5. set_params(params):
    - 설명: 모델의 매개변수를 설정합니다.
    - 인자:
        - `params`: 설정할 매개변수 딕셔너리
    - 리턴값: 설정된 `SVR` 객체
    - 예시: svr.set_params(C=10.0, epsilon=0.2)
...
print('')
```

In [20]: *#위 설명 예제*

...

이 예제에서는 보스턴 주택 가격 데이터 세트를 사용하여 SVR 모델을 학습시키고 예측을 수행합니다. 데이터는 `StandardScaler`를 사용하여 스케일링되며, SVR 모델의 커널은 기본값인 'rbf'로 설정되어 있습니다. 모델 학습 후, 테스트 데이터에 대한 예측을 수행하고 모델의 R^2 점수를 출력합니다.

```
from sklearn.svm import SVR
from sklearn.datasets import load_boston
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 데이터 로드
X, y = load_boston(return_X_y=True)

# 데이터 스케일링
scaler = StandardScaler()
X = scaler.fit_transform(X)

# 데이터 분할
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# SVR 모델 정의
svr = SVR(kernel='rbf', C=1.0, epsilon=0.1)

# 모델 학습
svr.fit(X_train, y_train)

# 예측
predictions = svr.predict(X_test)

# 성능 평가
score = svr.score(X_test, y_test)
print(f'R^2 score: {score}')
```

R^2 score: 0.6488085831738599

In [21]:

```
'''
코드는 다음과 같은 작업을 수행합니다.
1. `ColumnTransformer`를 사용하여 데이터 변환 파이프라인을 정의합니다. 이 파이프라인은 세 가지 변환 단계로 구성됩니다.
    - 첫 번째 단계(`std`): 주어진 열들(`TotalHour`, `lag_1` ~ `lag_6`)을 표준화한 후 PCA(주성분 분석)를 적용하여 차원을 5로 축소함
    이렇게 함으로써 데이터의 주성분을 추출하여 데이터의 차원을 줄입니다.
    - 두 번째 단계(`ohe`): 'DayName' 열을 가변수화(원-핫 인코딩)합니다.
    이 열은 각 요일을 나타내는 범주형 데이터이며, 가변수화를 통해 각 요일에 대한 이진 특성을 만듭니다.
```

- 세 번째 단계(`pt`): 'AM' 열은 그대로 유지됩니다.

2. `make\_pipeline`을 사용하여 데이터 변환 파이프라인과 SVR(Support Vector Regressor) 모델을 연결하여 파이프라인을 구성합니다. SVR 모델은 rbf 커널을 사용하고, gamma와 C 파라미터 값이 설정되어 있습니다.
3. `eval\_model` 함수를 사용하여 구성된 파이프라인 모델(`reg\_svr3`)을 평가합니다. 이 함수는 학습된 모델을 사용하여 검증 데이터에 대한 예측을 수행하고, 평균 절대 오차(Mean Absolute Error)를 계산하여 모델의 성능을 평가합니다.

따라서 이 코드는 주어진 데이터에 대해 PCA를 적용한 후 SVR 모델을 학습하고, 이를 사용하여 검증 데이터에 대한 예측을 수행하여 모델의 성능을 평가합니다.

SVR3: TotalHour와 lag1~6 까지를 PCA 를 적용합니다.

```
from sklearn.svm import SVR
from sklearn.decomposition import PCA

# make_pipeline는 ColumnTransformer 안에서도 쓸 수 있습니다.
# StandardScaler → PCA (n_components=5)
ct = ColumnTransformer([
    ('std', make_pipeline(StandardScaler(), PCA(n_components=5)),
     # TotalHour, lag_1 ~ 6 까지 표준화합니다.
     ['TotalHour'] + ['lag_{}'.format(i) for i in range(1, 7)]),
    # DayName을 가변수화 합니다.
    ('ohe', OneHotEncoder(categories='auto'), ['DayName']),
    ('pt', 'passthrough', ['AM'])
])

reg_svr3 = make_pipeline(ct, SVR(kernel='rbf', gamma=0.3, C=1000))
eval_model('svr + pca', reg_svr3)
```

124.66236, 31.23500

result

model_name

baseline 146.31594, 59.58739

svr + pca 124.66236, 31.23500

svr2 126.06277, 47.94924

```
In [22]: prd = select_model(reg_svr3)
print('자가채점:{}'.format(mean_absolute_error(s_kaggle_ans, prd)))
```

자가채점 : 32.703167616512054

```
In [23]: '''
`VotingRegressor`는 scikit-learn 라이브러리에서 제공하는 앙상블 회귀 기법 중 하나로,
여러 개의 다른 회귀 모델을 결합하여 예측 성능을 향상시키는 방법입니다.
이 모델은 개별 회귀 모델의 예측값의 평균을 계산하여 최종 예측값을 도출합니다.

### 인자 (Parameters)
`VotingRegressor`의 주요 인자는 다음과 같습니다:

1. estimators:
   - 설명: ('name', estimator) 형식의 튜플 리스트. 여기서 `name`은 각 개별 모델의 이름이고,
     `estimator`는 실제 회귀 모델입니다.
   - 타입: list of (str, estimator) tuples
   - 예시:
     estimators = [
         ('ridge', Ridge(alpha=1.0)),
         ('svr', SVR(kernel='linear')),
         ('rf', RandomForestRegressor(n_estimators=100))
     ]

2. weights:
   - 설명: 각 모델에 가중치를 부여하기 위한 리스트. 가중치는 모델의 중요도를 반영합니다.
   - 타입: list of float or None, default=None
   - 기본값: None (모든 모델에 동일한 가중치를 부여)
   - 예시: weights = [0.5, 0.2, 0.3]

3. n_jobs:
   - 설명: 계산에 사용할 병렬 작업의 수. -1로 설정하면 모든 프로세서를 사용합니다.
   - 타입: int, default=None
   - 기본값: None (단일 프로세서 사용)
   - 예시: n_jobs = -1

4. verbose:
   - 설명: 처리 과정의 로그 출력을 설정합니다.
   - 타입: bool, default=False
   - 기본값: False (로그 출력 없음)

### 리턴값 (Returns)
`VotingRegressor`는 별도의 리턴값을 제공하지 않지만, 이 클래스를 통해 생성된 객체는
여러 메서드를 통해 다양한 기능을 제공합니다. 주요 메서드는 다음과 같습니다:
```

```

1. fit(X, y):
    - 설명: 각 개별 회귀 모델을 학습시킵니다.
    - 인자:
        - `X`: 훈련 데이터 (특징 행렬)
        - `y`: 타겟 값 (레이블)
    - 리턴값: 학습된 `VotingRegressor` 객체
    - 예시: voting_regressor.fit(X_train, y_train)
2. predict(X):
    - 설명: 입력 데이터에 대한 예측값을 반환합니다.
    - 인자:
        - `X`: 예측할 데이터 (특징 행렬)
    - 리턴값: 예측값의 배열
    - 예시: predictions = voting_regressor.predict(X_test)
3. score(X, y):
    - 설명: 모델의 성능을 평가합니다.
    - 인자:
        - `X`: 테스트 데이터 (특징 행렬)
        - `y`: 실제 값 (레이블)
    - 리턴값: 성능 점수 (R^2 score)
    - 예시: score = voting_regressor.score(X_test, y_test)
...
print('')

```

In [24]: *# 위 설명 예제*

```

...
이 예제에서는 보스턴 주택 가격 데이터 세트를 사용하여 `Ridge`, `SVR`, `RandomForestRegressor` 세 개의 모델을
앙상블로 결합하여 예측을 수행합니다.
`VotingRegressor`를 통해 각 모델의 예측값의 가중 평균을 계산하여 최종 예측을 도출하고, 모델의 성능을 평가합니다.
...

from sklearn.linear_model import Ridge
from sklearn.svm import SVR
from sklearn.ensemble import RandomForestRegressor, VotingRegressor
from sklearn.datasets import load_boston
from sklearn.model_selection import train_test_split

# 데이터 로드
X, y = load_boston(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

```

```

# 개별 모델 정의
estimators = [
    ('ridge', Ridge(alpha=1.0)),
    ('svr', SVR(kernel='linear')),
    ('rf', RandomForestRegressor(n_estimators=100))
]

# VotingRegressor 객체 생성
voting_regressor = VotingRegressor(estimators=estimators, weights=[0.5, 0.2, 0.3])

# 모델 학습
voting_regressor.fit(X_train, y_train)

# 예측
predictions = voting_regressor.predict(X_test)

# 성능 평가
score = voting_regressor.score(X_test, y_test)
print(f'R^2 score: {score}')

```

R^2 score: 0.7698187838623853

In [25]:

```

'''
코드는 다음과 같은 작업을 수행합니다.

1. `VotingRegressor`를 사용하여 여러 개의 회귀 모델을 앙상블하는 모델인 `reg_vt`를 정의합니다.
   여기서는 두 개의 서로 다른 SVR 모델(`reg_svr2`와 `reg_svr3`)을 사용합니다.

2. `eval_model` 함수를 사용하여 앙상블된 모델인 `reg_vt`를 평가합니다.
   이 함수는 학습된 앙상블 모델을 사용하여 검증 데이터에 대한 예측을 수행하고, 평균 절대 오차(MAE)를 계산하여 모델의 성능을 평가합
따라서 이 코드는 SVR 모델의 앙상블인 `reg_vt`를 만들고, 이를 사용하여 검증 데이터에 대한 예측을 수행하여 모델의 성능을 평가합니다.
'''

from sklearn.ensemble import VotingRegressor

reg_vt = VotingRegressor([
    # ('baseline', reg_svr),
    ('svr2', reg_svr2),
    ('svr3', reg_svr3),
])
eval_model('vt1', reg_vt)

```


124.00514, 38.90055

result**model_name**

baseline 146.31594, 59.58739

svr + pca 124.66236, 31.23500

svr2 126.06277, 47.94924

vt1 124.00514, 38.90055

```
In [26]: prd = select_model(reg_vt)
print('자가채점:{}'.format(mean_absolute_error(s_kaggle_ans, prd)))
```

자가채점:28.71179298443789

```
In [27]: '''
`GradientBoostingRegressor`는 scikit-learn 라이브러리에서 제공하는 회귀 모델로, 그래디언트 부스팅 알고리즘을 구현한 것입니다.
이 모델은 여러 약한 학습기를 결합하여 강력한 예측 모델을 만드는 데 사용됩니다.
```

인자 (Parameters)

- **n_estimators**: 부스팅 단계를 수행할 트리의 수. 기본값은 100입니다.
- **learning_rate**: 각 트리의 기여도를 조절하는 인자. 기본값은 0.1입니다.
- **subsample**: 각 부스팅 단계에서 사용할 샘플의 비율. 기본값은 1.0입니다.
- **criterion**: 분할의 품질을 측정하는 기준. 기본값은 'friedman_mse'입니다.
- **min_samples_split**: 내부 노드를 분할하는 데 필요한 최소 샘플 수. 기본값은 2입니다.
- **min_samples_leaf**: 리프 노드에 필요한 최소 샘플 수. 기본값은 1입니다.
- **min_weight_fraction_leaf**: 리프 노드에 필요한 최소 샘플 가중치의 분수. 기본값은 0.0입니다.
- **max_depth**: 개별 회귀 추정기의 최대 깊이. 기본값은 3입니다.
- **min_impurity_decrease**: 분할을 실행하는 데 필요한 최소 불순도 감소. 기본값은 0.0입니다.
- **init**: 초기 추정기. 기본값은 None입니다.
- **random_state**: 난수 생성기 시드. 기본값은 None입니다.
- **max_features**: 분할에 사용할 최대 피처 수. 기본값은 None입니다.
- **alpha**: Huber 손실 함수와 퀴타일 손실 함수에 대한 알파. 기본값은 0.9입니다.
- **verbose**: 학습 과정의 출력 여부. 기본값은 0입니다.
- **max_leaf_nodes**: 리프 노드의 최대 수. 기본값은 None입니다.
- **warm_start**: 이전 호출의 솔루션을 재사용할지 여부. 기본값은 False입니다.
- **presort**: 훈련 데이터 정렬 여부. 기본값은 'deprecated'이며, True로 설정하면 성능이 향상될 수 있습니다.
- **validation_fraction**: 조기 종료를 위한 훈련 데이터의 비율. 기본값은 0.1입니다.
- **n_iter_no_change**: 조기 종료를 위한 허용 가능한 반복 횟수. 기본값은 None입니다.
- **tol**: 조기 종료를 위한 허용 오차. 기본값은 1e-4입니다.

- `ccp_alpha`: 비용 복잡성 가지치기 기준. 기본값은 `0.0`입니다.

리턴값 (Return Values)

`GradientBoostingRegressor` 객체 자체를 반환합니다. 이 객체는 `fit`, `predict`, `score` 등의 메서드를 통해 모델 학습 및 예측을 수행할 수 있습니다.

속성 (Attributes)

- `feature_importances_`: 피처 중요도를 나타내는 배열. 각 피처의 상대적 중요도를 평가합니다.
- `oob_improvement_`: 각 반복에 대한 OOB(out-of-bag) 개선 점수.
- `train_score_`: 각 반복에서 훈련 데이터에 대한 점수.
- `loss_`: 사용된 손실 함수.
- `init_`: 초기 추정기.
- `estimators_`: 각 부스팅 단계에서 학습된 회귀 추정기의 배열.
- `n_features_in_`: 훈련 데이터에서 사용된 피처 수.
- `n_outputs_`: 타겟의 출력 수.

함수 (Methods)

- `fit(X, y, sample_weight=None, monitor=None)`: 모델을 학습합니다.
 - `X`: 훈련 데이터.
 - `y`: 타겟 값.
 - `sample_weight`: 각 샘플의 가중치.
 - `monitor`: 콜백 함수.
- `predict(X)`: 입력 데이터에 대한 예측을 수행합니다.
 - `X`: 입력 데이터.
- `score(X, y, sample_weight=None)`: 모델의 R^2 점수를 반환합니다.
 - `X`: 테스트 데이터.
 - `y`: 실제 값.
 - `sample_weight`: 샘플 가중치.
- `staged_predict(X)`: 각 반복 단계별 예측을 생성합니다.
 - `X`: 입력 데이터.
- `apply(X)`: 각 입력 샘플이 리프 노드에 매핑된 인덱스를 반환합니다.
 - `X`: 입력 데이터.
- `decision_function(X)`: 입력 데이터에 대한 결정 함수 값을 반환합니다.
 - `X`: 입력 데이터.
- `get_params(deep=True)`: 현재 파라미터를 반환합니다.
 - `deep`: 모든 파라미터를 재귀적으로 반환할지 여부.
- `set_params(params)`: 파라미터를 설정합니다.
 - `params`: 설정할 파라미터.

예시 코드

```
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
```

```

from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_squared_error

# 데이터 생성
X, y = make_regression(n_samples=1000, n_features=20, noise=0.1)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 모델 초기화
model = GradientBoostingRegressor(n_estimators=100, max_depth=5, learning_rate=0.1)

# 모델 학습
model.fit(X_train, y_train)

# 예측
y_pred = model.predict(X_test)

# 평가
mse = mean_squared_error(y_test, y_pred)
print(f'Mean Squared Error: {mse}')

위 코드는 `GradientBoostingRegressor`를 사용하여 회귀 모델을 학습하고 평가하는 예시입니다. `
GradientBoostingRegressor`의 다양한 파라미터와 메서드를 통해 모델을 최적화할 수 있습니다.
'''

print('')

```

In [28]:

```

'''
코드는 다음과 같은 작업을 수행합니다:

1. 문자열 `X_gb`에 포함된 변수 목록을 정의합니다. 이 목록은 공백으로 분리된 변수 이름들을 포함하고 있습니다.
2. `ColumnTransformer`를 사용하여 데이터의 특정 열을 선택하고, 이를 회귀 모델에 입력으로 사용할 수 있도록 전처리합니다.
   여기서는 `pt`를 사용하여 지정된 변수들을 그대로 전달합니다.
3. Gradient Boosting 회귀 모델을 구성합니다.
   이 모델은 `GradientBoostingRegressor`를 사용하며, 다양한 하이퍼파라미터를 설정하여 세부 조정됩니다.
4. `eval_model` 함수를 호출하여 Gradient Boosting 회귀 모델을 평가합니다.
   이 함수는 모델을 학습하고 검증 데이터에 대한 예측을 수행하여 모델의 성능을 측정합니다.

1. 변수 목록 정의: 코드의 첫 번째 줄에서는 Gradient Boosting 회귀 모델에 사용할 변수 목록을 문자열 형태로 정의합니다.
   `split()` 함수를 사용하여 쉼표를 기준으로 문자열을 분할하고, `strip()` 함수를 사용하여 각 변수 이름 주위의 공백을 제거합니다.
   그 결과, `X_gb`는 변수 이름을 담은 리스트가 됩니다.
2. ColumnTransformer 구성: 두 번째 줄에서는 `ColumnTransformer`를 정의합니다.
'''

```

이를 통해 데이터의 특정 열을 선택하고 전처리할 수 있습니다. 여기서는 `pt`를 사용하여 지정된 변수들을 그대로 전달합니다. 즉, 추가적인 전처리 없이 입력 변수들이 그대로 모델에 입력됩니다.

3. Gradient Boosting 회귀 모델 구성:

세 번째 줄에서는 Gradient Boosting 회귀 모델을 구성합니다.

`GradientBoostingRegressor`를 사용하여 모델을 정의하고, 다양한 하이퍼파라미터를 설정합니다.

이 모델은 손실 함수로 `huber`를 사용하며, 학습률(learning rate), 트리 개수(n_estimators),

최대 깊이(max_depth), 최소 리프 샘플 수(min_samples_leaf) 등을 지정합니다.

4. 모델 평가:

마지막으로 `eval\_model` 함수를 호출하여 Gradient Boosting 회귀 모델을 평가합니다.

이 함수는 모델을 학습하고 검증 데이터에 대한 예측을 수행하여 모델의 성능을 측정합니다.

...

GB: Gradient Boosting Tree

```
from sklearn.ensemble import GradientBoostingRegressor
```

```
X_gb = "15분, 30분, 45분, 60분, DayName, Hour, AM, Weekend_yn, Holiday_yn, \
        Avg, TotalHour, 생산량, 기온, 풍속, 습도, 강수량, 전기요금(계절), 공장인원, 인건비"
```

```
X_gb = [i.strip() for i in X_gb.split(',')]
print('X_gb = ', X_gb)
```

별도의 전처리 없이 지정된 변수들을 사용합니다.

```
ct = ColumnTransformer([
    ('pt', 'passthrough', X_gb)
])
```

```
reg_gb = make_pipeline(
    ct,
    GradientBoostingRegressor(
        loss = 'huber', # 손실 함수: huber
        learning_rate=0.03,
        n_estimators = 300,
        max_depth = 6,
        min_samples_leaf = 4,
        random_state = 123
    )
)
eval_model('gb', reg_gb)
```

```
X_gb = ['15분', '30분', '45분', '60분', 'DayName', 'Hour', 'AM', 'Weekend_yn', 'Holiday_yn', 'Avg', 'TotalHour', '생산량', '기온', '풍속', '습도', '강수량', '전기요금(계절)', '공장인원', '인건비']
107.41606, 44.25745
```

result	
model_name	
baseline	146.31594, 59.58739
gb	107.41606, 44.25745
svr + pca	124.66236, 31.23500
svr2	126.06277, 47.94924
vt1	124.00514, 38.90055

In [29]:

```
'''
1. 변수 목록 정의:
    'X_rf' 문자열에는 랜덤 포레스트 모델에 사용할 예측 변수들의 목록이 포함되어 있습니다.
    각 변수는 쉼표로 구분되어 있으며, 이 목록은 문자열을 분할하여 리스트 형태로 저장됩니다.
2. ColumnTransformer 구성:
    'ColumnTransformer'는 입력 데이터를 변환하기 위해 사용됩니다.
    여기서는 'pt' 변환기를 사용하여 데이터의 모든 변수를 그대로 전달합니다.
3. 랜덤 포레스트 회귀 모델 구성:
    'RandomForestRegressor' 클래스를 사용하여 랜덤 포레스트 회귀 모델을 생성합니다.
    모델의 하이퍼파라미터로는 트리 개수(n_estimators), 트리의 최대 깊이(max_depth) 등이 설정되어 있습니다.
4. 모델 평가:
    'eval_model' 함수를 사용하여 랜덤 포레스트 회귀 모델을 평가합니다.
    이 함수는 모델을 학습하고 검증 데이터에 대한 예측을 수행한 후, 평균 절대 오차(MAE)를 계산하여 모델의 성능을 평가합니다.
...

# RF: Random Forest

from sklearn.ensemble import RandomForestRegressor

X_rf = "15분, 30분, 45분, 60분, DayName, Hour, AM, Weekend_yn, Holiday_yn, \
        Avg, TotalHour, 생산량, 기온, 풍속, 습도, 강수량, 전기요금(계절), 공장인원, 인건비"
X_rf = [i.strip() for i in X_rf.split(',')]
print('X_rf = ', X_rf)

ct = ColumnTransformer([
    ('pt', 'passthrough', X_rf)
])

reg_rf = make_pipeline(
```

```
ct,
RandomForestRegressor(n_estimators=50, max_depth=10, random_state=123)
)
eval_model('rf', reg_rf)
```

```
X_rf = ['15분', '30분', '45분', '60분', 'DayName', 'Hour', 'AM', 'Weekend_yn', 'Holiday_yn', 'Avg', 'TotalHour', '생산량', '기온', '풍속', '습도', '강수량', '전기요금(계절)', '공장인원', '인건비']
107.84414, 41.78158
```

result

model_name

model_name	score
baseline	146.31594, 59.58739
gb	107.41606, 44.25745
rf	107.84414, 41.78158
svr + pca	124.66236, 31.23500
svr2	126.06277, 47.94924
vt1	124.00514, 38.90055

```
In [30]: prd = select_model(reg_rf)
print('자가채점:{}'.format(mean_absolute_error(s_kaggle_ans, prd)))
```

자가채점:34.31395002304494

```
In [31]: '''
`XGBRegressor`는 XGBoost 라이브러리에서 제공하는 회귀 모델입니다.
XGBoost는 강력한 그래디언트 부스팅 알고리즘을 구현한 라이브러리로, 높은 성능과 유연성으로 잘 알려져 있습니다.
```

인자 (Parameters)

- `n_estimators`: 부스팅 라운드의 수. 기본값은 100입니다.
- `max_depth`: 트리의 최대 깊이. 트리의 복잡도를 제어합니다. 기본값은 6입니다.
- `learning_rate (eta)`: 각 부스팅 단계에서 가중치를 줄이는 요인. 기본값은 0.3입니다.
- `verbosity`: 출력되는 메시지의 세부 수준. 기본값은 1입니다.
- `objective`: 최적화할 목적 함수. 기본값은 'reg:squarederror'입니다.
- `booster`: 사용할 부스터 모델 (gbtree, gblinear, dart). 기본값은 'gbtree'입니다.
- `n_jobs`: 병렬 처리에 사용할 스레드 수. 기본값은 1입니다.
- `gamma`: 노드 분할에 필요한 최소 손실 감소. 기본값은 0입니다.
- `min_child_weight`: 자식 노드가 필요한 최소 가중치 합. 기본값은 1입니다.
- `subsample`: 각 부스팅 단계에서 사용할 훈련 데이터의 비율. 기본값은 1입니다.

- `colsample_bytree`: 각 트리를 구성할 때 사용할 피처의 비율. 기본값은 1입니다.
- `colsample_bylevel`: 각 트리 레벨에서 사용할 피처의 비율. 기본값은 1입니다.
- `colsample_bynode`: 각 노드에서 사용할 피처의 비율. 기본값은 1입니다.
- `reg_alpha`: L1 정규화 항. 기본값은 0입니다.
- `reg_lambda`: L2 정규화 항. 기본값은 1입니다.
- `scale_pos_weight`: 불균형 데이터셋의 경우, 양성 데이터 포인트에 대한 가중치 비율. 기본값은 1입니다.
- `base_score`: 모든 예측에 대한 초기 예측 값. 기본값은 0.5입니다.
- `random_state`: 랜덤 시드 값. 기본값은 0입니다.
- `missing`: 결측값 표시를 위한 값. 기본값은 NaN입니다.
- `importance_type`: 피처 중요도 계산 방법. 기본값은 'gain'입니다.

리턴값 (Return Values)

`XGBRegressor` 객체 자체를 반환합니다. 이 객체는 `fit`, `predict`, `score` 등의 메서드를 통해 모델 학습 및 예측을 수행할 수 있습니다.

속성 (Attributes)

- `feature_importances_`: 피처 중요도를 나타내는 배열. 피처의 중요도를 평가합니다.
- `best_iteration`: 최적의 반복 횟수.
- `best_score`: 최적의 점수.
- `booster`: 내부 Booster 객체.
- `evals_result_`: 평가 결과.

함수 (Methods)

- `fit(X, y, sample_weight=None, eval_set=None, eval_metric=None, early_stopping_rounds=None, verbose=True, xgb_model=None, sample_weight_eval_set=None, callbacks=None)`: 모델을 학습합니다.
 - `X`: 훈련 데이터.
 - `y`: 타겟 값.
 - `sample_weight`: 각 샘플의 가중치.
 - `eval_set`: 평가를 위한 검증 데이터.
 - `eval_metric`: 평가 지표.
 - `early_stopping_rounds`: 조기 종료 라운드 수.
 - `verbose`: 학습 과정의 출력 여부.
 - `xgb_model`: 기존의 XGBoost 모델.
 - `sample_weight_eval_set`: 검증 데이터의 가중치.
 - `callbacks`: 콜백 함수.
- `predict(X, output_margin=False, ntree_limit=None, validate_features=True, base_margin=None, iteration_range=None)`: 예측을 수행합니다.
 - `X`: 입력 데이터.
 - `output_margin`: 결정 함수 값을 반환할지 여부.
 - `ntree_limit`: 사용할 트리의 수.
 - `validate_features`: 피처 유효성 검사.
 - `base_margin`: 베이스 마진.

- `iteration_range`: 사용할 반복 범위.
- `score(X, y, sample_weight=None)`: 모델의 R^2 점수를 반환합니다.
 - `X`: 테스트 데이터.
 - `y`: 실제 값.
 - `sample_weight`: 샘플 가중치.
- `get_booster()`: 내부 `Booster` 객체를 반환합니다.
- `save_model(fname)`: 모델을 파일로 저장합니다.
 - `fname`: 저장할 파일 이름.
- `load_model(fname)`: 파일에서 모델을 불러옵니다.
 - `fname`: 불러올 파일 이름.
- `get_params(deep=True)`: 현재 파라미터를 반환합니다.
 - `deep`: 모든 파라미터를 재귀적으로 반환할지 여부.
- `set_params(params)`: 파라미터를 설정합니다.
 - `params`: 설정할 파라미터.

예시 코드

```
import xgboost as xgb
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

# 데이터 생성
X, y = make_regression(n_samples=1000, n_features=20, noise=0.1)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 모델 초기화
model = xgb.XGBRegressor(n_estimators=100, max_depth=5, learning_rate=0.1)

# 모델 학습
model.fit(X_train, y_train)

# 예측
y_pred = model.predict(X_test)

# 평가
```



```
mse = mean_squared_error(y_test, y_pred)
print(f'Mean Squared Error: {mse}')
```

위 코드는 `XGBRegressor`를 사용하여 회귀 모델을 학습하고 평가하는 예시입니다.
 `XGBRegressor`의 다양한 파라미터와 메서드를 통해 모델을 최적화할 수 있습니다.

```
print('')
```

In [32]:

```
'''
이 코드는 기계 학습을 사용하여 XGBoost 모델을 만들고 평가하는 과정을 나타냅니다.

1. 변수 설정:
  - `X_xgb` 변수는 사용할 특성(feature)들을 나타냅니다. 이 특성들은 모델 학습에 사용될 데이터의 열(column)을 나타냅니다.
    여기서는 시간 관련 특성(15분, 30분, 45분, 60분, DayName, Hour), 날씨 관련 특성(기온, 풍속, 습도, 강수량),
    그리고 다른 외부 요인들(전기요금(계절), 공장인원, 인건비)이 포함되어 있습니다.
  - `ColumnTransformer`는 다양한 열 변환을 수행하기 위해 사용됩니다.
    이 예에서는 'passthrough' 변환을 사용하여 특성을 그대로 전달합니다.

2. 파이프라인 설정:
  - `make_pipeline` 함수를 사용하여 여러 변환 단계를 하나의 파이프라인으로 묶습니다.
    이를 통해 모델 학습 및 평가를 간단하게 관리할 수 있습니다.
  - 파이프라인의 첫 번째 단계는 앞서 정의한 `ColumnTransformer`인 `ct`입니다.
    이 변환은 데이터를 적절한 형식으로 변환하여 모델에 입력할 수 있도록 준비합니다.
  - 두 번째 단계는 XGBoost 회귀 모델을 정의합니다. `xgb.XGBRegressor`를 사용하여 XGBoost 회귀 모델을 생성하고,
    하이퍼파라미터(n_estimators, learning_rate, max_depth)를 설정합니다.

3. 모델 평가:
  - `eval_model` 함수를 호출하여 모델을 평가합니다.
    이 함수는 모델의 성능을 측정하기 위해 테스트 데이터에 대한 예측을 생성하고, 이를 기반으로 평가 지표를 계산합니다.

이 코드는 XGBoost 회귀 모델을 생성하고, 이 모델의 성능을 평가하기 위해 데이터를 준비하고, 적절한 평가 지표를 계산합니다.
'''

# XGB: XGBoost

import xgboost as xgb

X_xgb = "15분, 30분, 45분, 60분, DayName, Hour, AM, Weekend_yn, Holiday_yn, \
        Avg, TotalHour, 생산량, 기온, 풍속, 습도, 강수량, 전기요금(계절), 공장인원, 인건비"
X_xgb = [i.strip() for i in X_xgb.split(',')]
print('X_xgb = ', X_xgb)
```

```
ct = ColumnTransformer([
    ('pt', 'passthrough', X_xgb)
])

reg_xgb = make_pipeline(
    ct,
    xgb.XGBRegressor(n_estimators=500, learning_rate=0.02, max_depth=6, random_state=123)
)

eval_model('xgb', reg_xgb)
```

```
X_xgb = ['15분', '30분', '45분', '60분', 'DayName', 'Hour', 'AM', 'Weekend_yn', 'Holiday_yn', 'Avg', 'TotalHour', '생산량', '기온', '풍속', '습도', '강수량', '전기요금(계절)', '공장인원', '인건비']
106.29032, 45.38133
```

result**model_name**

baseline	146.31594, 59.58739
gb	107.41606, 44.25745
rf	107.84414, 41.78158
svr + pca	124.66236, 31.23500
svr2	126.06277, 47.94924
vt1	124.00514, 38.90055
xgb	106.29032, 45.38133

```
In [33]: prd = select_model(reg_xgb)
print('자가채점:{}'.format(mean_absolute_error(s_kaggle_ans, prd)))
```

자가채점 :44.10046746141167

```
In [34]: '''
코드는 선형 회귀(Linear Regression) 모델을 생성하고 평가하는 과정을 나타냅니다.

1. 변수 설정:
- `ct` 변수는 열 변환(Column Transformation)을 정의합니다.
  이를 통해 데이터의 특성을 적절하게 변환하여 모델에 입력할 수 있도록 합니다.
- 첫 번째 변환(`('ohe', OneHotEncoder(drop='first', categories='auto'), ['DayName'])`)은
```

- 'DayName' 열에 대해 원-핫 인코딩(One-Hot Encoding)을 수행합니다.
- 이 때, 첫 번째 카테고리를 삭제(drop='first')하여 다중 공선성을 방지합니다.
- 두 번째 변환(`('std', StandardScaler(),
['TotalHour'] + ['lag_{}'.format(i) for i in range(1, 7)])`)은
'TotalHour'와 'lag_1'부터 'lag_6'까지의 열에 대해 표준화(Standardization)를 수행합니다.
- 세 번째 변환(`('pt', 'passthrough', ['AM'])`)은 'AM' 열을 그대로 전달합니다.

2. 파이프라인 설정:

- `make\_pipeline` 함수를 사용하여 여러 변환 단계를 하나의 파이프라인으로 묶습니다.
- 이 파이프라인은 첫 번째로 정의된 열 변환 `ct`를 거쳐서 데이터를 변환한 후, 선형 회귀 모델(LinearRegression)을 적용합니다.

3. 모델 평가:

- `eval\_model` 함수를 호출하여 선형 회귀 모델을 평가합니다.
- 이 함수는 모델의 성능을 측정하기 위해 테스트 데이터에 대한 예측을 생성하고, 이를 기반으로 평가 지표를 계산합니다.

이렇게 설정된 코드는 주어진 데이터에 대해 선형 회귀 모델을 학습하고, 그 성능을 평가하는 과정을 나타냅니다.

```
...
```

```
from sklearn.linear_model import LinearRegression
```

```
# LinearRegression
```

```
ct = ColumnTransformer([
    # 'DayName' OneHotEncoder drop='first'
    ('ohe', OneHotEncoder(drop='first', categories='auto'), ['DayName']),
    # 'TotalHour', lag_1 ~ 6 StandardScaler
    ('std', StandardScaler(), ['TotalHour'] + ['lag_{}'.format(i) for i in range(1, 7)]),
    # 'AM' 'passthrough'
    ('pt', 'passthrough', ['AM'])
])
```

```
reg_lr = make_pipeline(ct, LinearRegression())
eval_model('lr', reg_lr)
```

```
164.29570, 85.11019
```

result	
model_name	
baseline	146.31594, 59.58739
gb	107.41606, 44.25745
lr	164.29570, 85.11019
rf	107.84414, 41.78158
svr + pca	124.66236, 31.23500
svr2	126.06277, 47.94924
vt1	124.00514, 38.90055
xgb	106.29032, 45.38133

In [35]:

'''

코드는 여러 모델의 예측을 결합하여 더 나은 예측을 만들기 위해 보팅(Voting)을 수행하는 과정을 나타냅니다.

1. VotingRegressor 설정:

- `VotingRegressor`는 여러 개의 회귀 모델을 결합하여 하나의 모델을 형성합니다.
이 모델은 여러 모델들의 예측을 평균화하여 더 나은 예측을 제공합니다.
- `reg\_vt2`는 `VotingRegressor`를 생성하는데 사용되는 객체입니다. 이 모델은 여러 개의 기반 모델들을 포함하고 있습니다.
- `VotingRegressor`의 매개변수로는 리스트 형태로 기반 모델들의 이름과 객체를 지정합니다.
여기서는 선형 회귀 모델(`reg\_lr`), 서포트 벡터 머신 회귀 모델(`reg\_svr3`), 그라디언트 부스팅 회귀 모델(`reg\_gb`), 랜덤 포레스트 회귀 모델(`reg\_rf`), XGBoost 회귀 모델(`reg\_xgb`)을 포함하고 있습니다.
- 각 모델은 주석처리로 제외되어 있는 것을 볼 수 있습니다. 이는 모델 선택의 여지를 남겨두는 것입니다.

2. 모델 평가:

- `eval\_model` 함수를 호출하여 보팅 모델(`reg\_vt2`)을 평가합니다.
이 함수는 모델의 성능을 측정하기 위해 테스트 데이터에 대한 예측을 생성하고, 이를 기반으로 평가 지표를 계산합니다.

이렇게 설정된 코드는 다양한 기반 모델의 예측을 결합하여 보팅(Voting)을 수행하고, 그 성능을 평가하는 과정을 나타냅니다.

'''

VotingRegressor

from sklearn.ensemble import VotingRegressor

Voting할 모델을 선별합니다.

```
reg_vt2 = VotingRegressor([
    #('baseline', reg_svr),
    #('svr2', reg_svr2),
    ('svr+pca', reg_svr3),
    ('gb', reg_gb),
    ('rf', reg_rf),
    ('xgb', reg_xgb),
    # ('lr', reg_lr)
])

eval_model('vt2', reg_vt2)
```

107.57788, 38.37770

result	
model_name	
baseline	146.31594, 59.58739
gb	107.41606, 44.25745
lr	164.29570, 85.11019
rf	107.84414, 41.78158
svr + pca	124.66236, 31.23500
svr2	126.06277, 47.94924
vt1	124.00514, 38.90055
vt2	107.57788, 38.37770
xgb	106.29032, 45.38133

```
In [36]: # 결과상 최종모델은 xgb를 선택하 것이지만, vt2의 자가 채점 결과를 봅니다.
# Voting 모델의 성능을 측정
prd = select_model(reg_vt2)
print('자가채점:{}'.format(mean_absolute_error(s_kaggle_ans, prd)))
```

자가채점 :33.41597463010524

In []: